

02-614 Week 5

String Algorithms

Aidan Jan

September 24, 2026

Suffix Trees

Similar to the tree in Aho-Corasick, but it stores all the suffixes of a string (including an ending character, \$). The following is an example for **banana\$**.

```
banana$
 anana$
  nana$
   ana$
    na$
     a$
      $
```

[INSERT IMAGE OF KEYWORD TREE] This is not quite a suffix tree yet. The main issue is that the keyword tree (aka. suffix Trie) has a size quadratic to the size of the string. So instead, we can compress it. [INSERT IMAGE OF SUFFIX TREE] We can make a further optimization where we represent all the substrings using string indices instead. [INSERT IMAGE OF SUFFIX TREE (COMPRESSED)] A **Suffix Tree** is a path-compressed keyword tree of all suffixes of A , with edges labeled with ranges in string A .

Some facts about the suffix tree

- The size of the suffix tree of T is $O(|T|)$
 - Additionally, a suffix tree has $O(|T|)$ leaves, where $|T|$ is the number of characters in the string. This is because every leaf is a \$, where there are T of.
- The suffix tree for T can be created in linear time ($O(|T|)$). (Ukkonen's Algorithm).

Uses for Suffix Tree

Given a suffix tree T for string S :

- Find number of occurrences of P in S
 - Follow the pattern down the tree in T , and count the number of leaves below it. $O(|T|)$ for suffix tree build, $O(|P|)$ to walk down P , $O(z)$ is number of output paths
- Longest repeat in S ?
 - Find the deepest node with two branches.
- Longest common substring between S and T ?
 - Create a suffix tree for $S\#T$ (concatenate the two strings with a character in the middle)
 - Color the leaves based on whether the suffixes begin in S or T ; any node that has branches leading to both colored leaves must be suffixes of both strings.
 - Find the deepest node leading to both colored leaves.

Lempel-Ziv Compression

This guy invented the `gzip` package for linux, which is widely used.

- This is a compression problem, where we have an input of string T , and output $\tilde{T}$, where **most** of the time, $|\tilde{T}| \leq |T|$, but (always) $\tilde{T} \rightarrow T$.

Algorithm:

- We walk from the left of the string to the right. Suppose we are position i in T .
- We find the longest substring starting at i that matches a substring contained in $T[1 : i - 1]$, inclusive. We output the location of the matched substring and its length.
 - If there is no matched substring, output $(T[i], 0)$
 - A given segment might contain chains of pointers.
- After encoding at position i , we move the character pointer to the end of the substring.
- To decode, start from the beginning of the string, and expand the pointers in order. Since each pointer points backwards, by the time we get to a given pointer, all the characters it points to must already be resolved.

How to Match Substrings

Lempel-Ziv compression requires us to find the longest substring from the given position i . We do this by using a suffix tree.

- Build the suffix tree for T , and label all of the leaves by the index they start.
- Let v be a node in T that splits into multiple branches (i.e., it is a longest common substring for at least two substrings.). In this case, v would be a good candidate for a matching substring for the non-smallest-indexed leaf.
- Label every branching node with $c(v) = \text{earliest suffix represented by } v$
- To search for matching substrings, walk down the tree following T after i . The match is the deepest node on that path that satisfies $c(v) + \text{depth}(v) < i$
- This runs in $O(|T|)$ because even though the tree traversal takes longer than linear, it is amortized linear because characters we traverse through in the tree are skipped once matched in the string. In other words, every character in the string is only visited once.

Longest Common Extension

We have two strings, S and T . Later, queries of the form (i, j) will be provided. Our output should be the length of the longest substring starting at $i \in S$ and $j \in T$.

- We can do this with a generalized suffix tree. That is, a suffix tree for string $S\%T$.
- A query (i, j) would be represented by two separate leaves S_i and T_j on the suffix tree, and their lowest common ancestor (i.e., deepest branching point) would be the solution.
- This is because the two matching substrings beginning at the two characters must share a common prefix.
- Algorithm takes $O(|T| + |S|)$ to build the tree, and $O(1)$ to answer LCE if all the pairings are stored in a lookup matrix.

Fact: Any tree can be preprocessed in linear time and linear space to answer $\text{LCA}(u, v)$ queries in $O(1)$ time. (Gabow, Tarjan 1983). You don't have to calculate the entire matrix, but rather store it implicitly.

Palindrome Finding Path

Input is string T . The goal is for every position of T , give the lengths of the longest **even** palindrome centered at that path.

- We can do this quickly and easily by creating a suffix tree containing T and its reverse, T^R .
- Now, run LCE on these two strings. $LCE_{T,T^R}(i, |T| - i - 1)$. The output is the answer.
- This runs in $O(|T|)$. Preprocessing the string in both directions is $O(|T|)$, running LCE at a given position is $O(1)$ after linear preprocessing, and there are $|T|$ queries.

K-mismatch Search

Our input is a string T , a pattern P , and an integer k . We want to find each position in T where P occurs with $\leq K$ substitutions.

- Again, this uses LCE. Search LCE at position i in T and position 1 in P .
- If there is a mismatch after x characters, then repeat LCE starting from $T[i + x + 1]$ and $P[x]$. We repeat this until the end of the pattern is reached, in which case we output the string index, or we stop after k mismatches.
- This runs in $O(k|T|)$ since each LCE is constant time, and for every position of the string we perform at most k LCE queries.

Longest Common Substring with Online Queries

We are given an input T and queries $P_1, P_2, \dots$. For each query P_i , we want the longest common substring between P_i and T .

- This one is slightly more involved because we can't create suffix tree containing every pattern. (They are given one at a time.)
- For this, we will need suffix links.

Suffix Links

For each node v in the suffix tree that represents $x\alpha$, $S\ell(v) \rightarrow$ node representing α . (x is a single character, α is a string.)

- If v is the node representing a single character (i.e., x), then its suffix link goes to the root.
- Provided with Ukkonen's algorithm during the construction of the suffix tree, also in linear time.

Back to the longest common substring with online queries algorithm:

- First, trace P in T . (and stop when the pattern no longer has a branch in the tree)
- If we have not reached the end of the string yet, follow a suffix link, and continue moving down.
- Record the deepest node reached with this method.
- Following a suffix link is equivalent to starting the string matching search one character later.
- This algorithm runs in $O(|P_i|)$ for every pattern.